Helm Workshop

2026-04-21

Introduction

This Helm workshop is designed to help you make better use of Helm in your Kubernetes deployments.

Prerequisites

This workshop assumes the following:

- You have basic familiarity with what Helm is and what it does.
- You have experience running a Helm chart in your own Kubernetes cluster.

If you are new to Helm, we recommend reviewing the official Helm documentation before proceeding.

Expectations

After completing this workshop, you will be able to:

- Create and install a Helm chart.
- Perform basic and advanced templating of resources.
- Package a Helm chart as a set of files or an OCI artifact.

Module 1: Background and Review

Content

POP QUIZ!

Try to answer each question before expanding the section to see how strong your current understanding of Helm is.

What is Helm and what problem does it solve?

▼ *Answer*

Helm is a package manager for Kubernetes that simplifies the deployment and management of applications.
It solves several key problems:

- **Complexity**: Kubernetes manifests can be complex and repetitive.
  Helm packages these manifests into reusable charts.
- **Configuration Management**: Helm provides a templating system that allows you to customize deployments without modifying the original manifests.
- **Versioning**: Helm tracks versions of your deployments, making it easier to roll back to previous configurations.
- **Dependency Management**: Helm can manage dependencies between different Kubernetes resources.

What are the main components of a Helm chart?

▼ *Answer*

A Helm chart consists of several key components including:

- **Chart.yaml**: Contains metadata about the chart, including its name, version, and description.
- **values.yaml**: Contains default configuration values that can be overridden during installation or upgrade.
- **templates/**: A directory containing Kubernetes manifest templates written in Go template syntax.
- **charts/**: An optional directory containing chart dependencies.
- **README.md**: Documentation explaining how to use the chart.

What is the difference between a Helm chart and a Helm release?

▼ *Answer*

A **Helm chart** is a collection of files that describe a set of Kubernetes resources.
It is a package that can be versioned, shared, and reused.
A chart is like a class in programming.

A **Helm release** is an instance of a chart that has been deployed to a Kubernetes cluster.
When you install a chart, Helm creates a release with a unique name.
You can have multiple releases of the same chart, each with different configurations.
A release tracks the state of the deployed resources and allows you to upgrade, rollback, or uninstall the resources.

What is the purpose of values.yaml in a Helm chart?

▼ *Answer*

The `values.yaml` file serves as the default configuration file for a Helm chart.
It defines default values that are used when rendering the templates in the chart.

Key purposes include:

- **Default Configuration**: Provides sensible defaults for all configurable parameters in the chart.
- **Documentation**: Acts as documentation by showing all available configuration options and their default values.
- **Customization**: Allows users to override these defaults when installing or upgrading a release, either by providing a custom values file or using the `--set` flag.
- **Separation of Concerns**: Separates configuration from template logic, making charts more maintainable and easier to customize.

What is a Helm repository and how is it used?

▼ *Answer*

A Helm repository is a location where Helm charts are stored and can be shared.
It is typically an HTTP server that serves an `index.yaml` file containing metadata about available charts.

Helm repositories are used to:

- **Share Charts**: Distribute charts to teams or the public.
- **Version Management**: Track multiple versions of charts in a centralized location.
- **Discovery**: Make it easy to find and install charts using `helm search` commands.

To use a repository, you add it with `helm repo add <name> <url>`, update the local cache with `helm repo update`, and then install charts from it using `helm install <release-name> <repo-name>/<chart-name>`.

What is the difference between `helm install` and `helm upgrade`?

▼ *Answer*

`helm install` is used to create a new release by installing a chart for the first time.
It deploys the chart's resources to the Kubernetes cluster and creates a new release record.

`helm upgrade` is used to update an existing release to a new version of a chart or with new configuration values.

It modifies the existing resources in the cluster based on the differences between the current and desired state.

Key differences:

- **Install** creates a new release; **upgrade** modifies an existing one.
- **Install** requires a unique release name; **upgrade** uses the name of an existing release.
- **Upgrade** can use the `--install` flag to create a release if it doesn't exist, combining both operations.

What is a Helm template and how does templating work?

▼ *Answer*

A Helm template is a Kubernetes manifest file that contains placeholders written in Go template syntax.
These placeholders are replaced with actual values when Helm renders the chart.

Templating allows you to:

- **Parameterize Resources**: Use variables like `{{ .Values.replicas }}` to make resource counts configurable.
- **Conditional Logic**: Use `{{ if }}` statements to include or exclude resources based on configuration.
- **Loops**: Use `{{ range }}` to create multiple resources from a list.
- **Access Built-in Objects**: Access information like release name, namespace, and chart metadata using objects like `{{ .Release.Name }}` and `{{ .Chart.Name }}`.

When you run `helm install` or `helm upgrade`, Helm processes all templates in the `templates/` directory, replaces the placeholders with values from `values.yaml` and any overrides, and sends the rendered manifests to Kubernetes.

How are dependencies defined and managed in Chart.yaml?

▼ *Answer*

Dependencies in Helm charts are defined in the `dependencies` section of the `Chart.yaml` file.
Each dependency specifies another chart that must be installed before the current chart can be deployed.

A dependency entry includes:

- **Name**: The name of the dependent chart.
- **Version**: The version or version range of the chart to use (e.g., "1.2.3" or "~1.2.0").
- **Repository**: The URL of the Helm repository where the chart can be found, or "file://" for local chart dependencies.
- **Condition**: An optional condition that determines whether the dependency should be installed (useful for optional dependencies).

- **Tags**: Optional tags that can be used to group dependencies and control their installation.

To use dependencies, you run `helm dependency update`, which downloads the specified charts into the `charts/` directory.
Helm then installs these dependencies before installing the main chart, ensuring that all required resources are available.
Dependencies are installed as sub-charts and can be accessed in templates using the `.Chart.Dependencies` object.

How does Helm manage rollbacks and what information does it track?

▼ *Answer*

Helm tracks the history of each release, storing information about every install, upgrade, and rollback operation.
This history includes:

- **Release Version**: Each operation increments the release revision number.
- **Chart Version**: The version of the chart that was deployed.
- **Configuration**: The values that were used for that revision.
- **Status**: Whether the release was successful, pending, or failed.
- **Description**: Optional notes about what changed in that revision.

To rollback, you use `helm rollback <release-name> <revision-number>`, which reinstalls the resources from the specified revision.
Helm maintains this history in Kubernetes secrets, allowing you to see all revisions with `helm history <release-name>` and revert to any previous state if needed.

If you could not answer all of the questions, that is perfectlyokay.
We will be reviewing these concepts and exploring more advanced topics as we progress through the workshop.

Module 2: Helm CLI

Content

Why Learn the Helm CLI?

While Helm is most often used as a client within a GitOps framework and not executed directly in production environments, understanding how to test Helm commands locally will significantly speed up the development process for charts.
By testing commands locally before committing changes to your GitOps workflow, you can quickly validate chart configurations, catch errors early, and iterate more efficiently on your Helm charts.

Creating and Deploying Your Chart: A Workflow

Let us walk through the typical workflow of creating a Helm chart, packaging it, pushing it to a registry, and installing it in a cluster.
This narrative will guide you through the commands you will use at each step.

Step 1: Creating Your Chart

First, you create a new Helm chart using the `helm create` command.
This generates a basic chart structure with sample templates and a `values.yaml` file:

```
helm create my-chart
```

This command creates a directory called `my-chart` with all the necessary files for a Helm chart.
You can now customize the templates and values to match your application requirements.

Step 2: Testing Your Chart Locally

Before packaging your chart, you should test it to ensure it renders correctly.
Use the `helm template` command to render your chart templates without installing anything:

```
helm template my-release ./my-chart
```

This command shows you exactly what Kubernetes resources will be created when you install the chart.
Review the output to verify that your templates are working as expected.

Step 3: Packaging Your Chart

Once you are satisfied with your chart, package it into a `.tgz` archive:

```
helm package ./my-chart
```

This creates a file named `my-chart-0.1.0.tgz` (assuming version 0.1.0 in your `Chart.yaml`).
The packaged chart is now ready to be distributed.

Step 4: Logging In to Your Registry

Before you can push your chart to a container registry, you need to authenticate.
Use the `helm registry login` command to log in to your registry:

```
helm registry login <registry-url>
```

For example, if you are using a container registry:

```
helm registry login registry.example.com
```

You will be prompted for your username and password or token.
Once authenticated, Helm can push charts to the registry.

Step 5: Pushing Your Chart

Now you can push your packaged chart to the registry as an OCI artifact:

```
helm push my-chart-0.1.0.tgz oci://registry.example.com/charts
```

This uploads your chart to the registry where it can be accessed by others or by your GitOps tools.

Step 6: Fixing and Pushing Again

If you discover an issue with your chart after pushing it, you can make corrections and push again.
After updating your chart files, increment the version in `Chart.yaml`, then package and push again:

```
helm package ./my-chart
helm push my-chart-0.2.0.tgz oci://registry.example.com/charts
```

Each version is stored separately in the registry, allowing you to maintain a history of your chart versions.

Step 7: Installing Your Chart

With your chart now available in the registry, you can install it in your Kubernetes cluster.
First, if needed, add the registry as a repository:

```
helm repo add my-registry oci://registry.example.com/charts
helm repo update
```

Then install the chart:

```
helm install my-release oci://registry.example.com/charts/my-chart --version 0.2.0
```

This command creates a new Helm release named `my-release` using your chart from the registry.

Step 8: Verifying Your Running Release

After installation, verify that your release is running correctly.
Check the status of your release:

```
helm status my-release
```

This shows detailed information about the release, including the status of all deployed resources.

You can also list all releases in your namespace:

```
helm list
```

This displays all Helm releases, their status, and when they were last updated.
Your `my-release` should appear in this list, confirming that it is installed and running in your cluster.

Next Steps

Now that you have an overview of the Helm CLI workflow, you will use these commands in the exercises section to explore them in greater detail.
The exercises will provide hands-on practice with each command, helping you build confidence and familiarity with the Helm CLI as you work through real-world scenarios.

Exercises

Exercise 0: Setting Up Helm Autocomplete for PowerShell

Before you begin working with Helm commands, set up autocomplete to make your workflow more efficient.
Helm provides autocomplete support for PowerShell that will help you discover commands and options as you type.

Step 1: Generate the Helm Autocomplete Script

Generate the PowerShell autocomplete script for Helm:

```
helm completion powershell | Out-File -FilePath $PROFILE -Append
```

This command generates the Helm autocomplete configuration and appends it to your PowerShell profile.

The `$PROFILE` variable points to your PowerShell profile file, which is loaded every time you start PowerShell.

Step 2: Reload Your PowerShell Profile

If you are currently in a PowerShell session, reload your profile to activate the autocomplete:

```
. $PROFILE
```

Alternatively, close and reopen your PowerShell window to load the updated profile.

Step 3: Verify Autocomplete is Working

Test that autocomplete is working by typing `helm ` (with a space) and pressing the Tab key.
You should see available Helm commands appear as suggestions.
You can continue pressing Tab to cycle through the available options.

Autocomplete will now help you discover Helm commands, subcommands, and flags as you work through the exercises.

Exercise 1: Creating Charts and Working with Repositories

In this exercise, you will create a Helm chart, explore its structure, work with repositories, and create a chart using a starter template.

Step 1: Create a New Chart

Create a new Helm chart named `my-first-chart`:

```
helm create my-first-chart
```

Step 2: List All of the Files Created

List all of the files that were created in the chart directory:

```
Get-ChildItem -Path .\my-first-chart -Recurse
```

This command shows you the complete directory structure of your new chart, including all subdirectories and files.

Step 3: Delete the Chart

Remove the chart directory that you just created:

```
Remove-Item -Path .\my-first-chart -Recurse -Force
```

Step 4: Add the Jetstack Repository

Add the Jetstack Helm repository to your local Helm configuration:

```
helm repo add jetstack https://charts.jetstack.io
```

Step 5: Pull the Jetstack Chart

Update your repository information and pull a chart from the Jetstack repository.
First, update your local repository cache:

```
helm repo update
```

Then, pull a chart from the Jetstack repository (for example, the cert-manager chart):

```
helm pull jetstack/cert-manager
```

This downloads the chart package to your current directory.

Step 6: Create a New Helm Chart with Jetstack as the Starter

Create a new Helm chart using a Jetstack starter template.
First, you may need to pull the starter chart or use it directly:

```
helm create my-jetstack-chart --starter jetstack/cert-manager
```

Alternatively, if you have a local starter, you can specify the path to the starter chart.

This creates a new chart based on the Jetstack starter template, giving you a foundation that follows Jetstack's chart structure and best practices.

Exercise 2: Finding RBAC Settings for cert-manager

In this exercise, you will explore the cert-manager Helm chart to find its RBAC (Role-Based Access Control) settings.
You will examine the default values and use templating to see what RBAC resources would be created.

Step 1: Add the Jetstack Repository

If you have not already added the Jetstack repository, add it now:

```
helm repo add jetstack https://charts.jetstack.io
```

Update your repository cache to ensure you have the latest chart information:

```
helm repo update
```

Step 2: Show What the Default Values Are

View the default values for the cert-manager chart:

```
helm show values jetstack/cert-manager
```

This command displays all the default configuration values for the cert-manager chart.
Look through the output to find RBAC-related settings, which are typically under sections like `rbac`, `serviceAccount`, or `global.rbac`.

Step 3: Template the Install with Default Values

Use the `helm template` command to render the cert-manager chart with default values and examine the RBAC resources that would be created:

```
helm template cert-manager jetstack/cert-manager
```

This command renders all the Kubernetes manifests that would be created when installing cert-manager.
Search through the output for RBAC resources such as:
* `ServiceAccount` resources
* `ClusterRole` resources
* `ClusterRoleBinding` resources
* `Role` resources
* `RoleBinding` resources

You can also filter the output to show only RBAC-related resources by piping the output through `Select-String`:

```
helm template cert-manager jetstack/cert-manager | Select-String -Pattern "kind:
(ServiceAccount|ClusterRole|ClusterRoleBinding|Role|RoleBinding)"
```

This helps you quickly identify all RBAC-related resources that the chart would create.

Exercise 3: Exploring Local Helm Configuration

In this exercise, you will explore Helm's local configuration and settings.
You will examine Helm's environment variables and inspect the structure of registry and repository configuration files.

Step 1: Identify Helm's Current Settings

Use the `helm env` command to display Helm's current environment settings:

```
helm env
```

This command shows important Helm configuration paths and settings, including:
* The location of Helm's configuration directory
* The location of the cache directory
* The location of the data directory
* Plugin and registry paths

These paths tell you where Helm stores its configuration files and cached data on your system.

Step 2: Show the Structure of the Repository Configuration

Helm stores repository information in a configuration file.
First, identify where Helm stores its configuration by checking the output from `helm env`, or use the default location.
On Windows, this is typically in your user profile directory.

View the repository configuration file:

```
Get-Content $env:USERPROFILE\.config\helm\repositories.yaml
```

If the file does not exist at that location, you can find the exact path using:

```
helm env | Select-String "HELM_REPOSITORY_CONFIG"
```

This shows you the structure of your repository configuration, including all repositories you have added, their URLs, and metadata.

Step 3: Show the Structure of the Registry Configuration

Helm also maintains configuration for OCI registries.
View the registry configuration file:

```
Get-Content $env:USERPROFILE\.config\helm\registry.json
```

Alternatively, find the exact path using:

```
helm env | Select-String "HELM_REGISTRY_CONFIG"
```

This file contains authentication and configuration information for OCI registries that you have logged into.
It may include credentials, registry URLs, and other registry-specific settings.

You can also explore the Helm configuration directory structure:

```
Get-ChildItem -Path $env:USERPROFILE\.config\helm\ -Recurse
```

This shows you all Helm configuration files and directories, giving you a complete view of Helm's local configuration structure.

Module 3: File Structure and Templating

Content

For detailed information about Helm chart structure and templating, refer to the official Helm documentation.

This module provides a brief overview of the essential concepts that you will use in the exercises.

Chart Structure

A Helm chart follows a standard directory structure.
When you create a chart using `helm create`, it generates the following structure:

```
mychart/
├─── Chart.yaml          # Chart metadata
├─── values.yaml        # Default configuration values
├─── charts/            # Chart dependencies (optional)
├─── templates/         # Template files
│    ├─── deployment.yaml
│    ├─── service.yaml
│    └─── ...
└─── templates/NOTES.txt # Installation notes (optional)
```

Key Files and Directories

- **Chart.yaml**: Contains metadata about the chart, including name, version, and dependencies.
- **values.yaml**: Defines default configuration values that can be overridden during installation.
- **templates/**: Contains Kubernetes manifest templates written in Go template syntax.
  These templates are rendered with values from `values.yaml` and any overrides provided during installation.
- **charts/**: Contains chart dependencies that are downloaded when you run `helm dependency update`.

The Power of Templating: Taking Helm to the Next Level

The whole point of Helm is to prevent repetition.
Instead of maintaining multiple nearly identical Kubernetes manifests for different environments or configurations, you create a single template that can be customized through values.

Understanding how templating, functions, flow control, and syntax work together is what separates basic Helm users from advanced practitioners.
These concepts enable you to:

- **Eliminate Duplication**: Create reusable templates that adapt to different scenarios without copying and modifying files.

- **Build Flexible Charts**: Design charts that can handle various deployment patterns, environments, and configurations through intelligent templating.
- **Maintain Consistency**: Ensure that all deployments follow the same patterns and best practices while allowing necessary customization.
- **Scale Your Knowledge**: Apply advanced patterns like template helpers, named templates, and complex conditional logic to solve sophisticated deployment challenges.

Mastering templating syntax, understanding Helm's built-in functions, leveraging flow control structures, and knowing when and how to use each feature will take your Helm chart development to the next level.
The exercises in this module will give you hands-on practice with these advanced concepts.

Critical Resources for Template Design

When designing Helm templates, you will frequently reference two essential documentation resources:

- Built-in Objects: This documentation describes all the objects available in templates, including `Release`, `Values`, `Chart`, `Files`, `Capabilities`, and `Template`.
  These objects provide access to release information, chart metadata, file contents, and cluster capabilities.
- Template Function List: This comprehensive reference lists all available template functions organized by category, including logic and flow control, string manipulation, dictionaries, lists, math operations, and many more.

These resources are critical for designing effective templates.
Bookmark them and refer to them regularly as you work through the exercises and develop your own charts.

Creating Reusable Template Libraries

You can extend the principle of eliminating repetition beyond individual charts to your entire organization by creating shared template libraries.

Library Charts with Template Helpers

You can create Helm charts that contain only template helpers and no actual Kubernetes resources.
These library charts serve as repositories of reusable template functions, named templates, and common patterns that can be shared across multiple charts in your organization.

A library chart typically contains:
* Template files with named templates (using `{{- define }}` blocks)
* Helper functions for common operations
* Standardized patterns for labels, annotations, resource naming, and other organizational conventions

These templates are not rendered as Kubernetes resources when the library chart is included as a dependency.

Instead, they become available for use in your application charts through the `{{ include }}` function or by referencing the named templates directly.

Using Library Charts as Dependencies

By declaring library charts as dependencies in your `Chart.yaml`, you can create a uniform approach across your team.
When you add a library chart as a dependency, all of its template helpers become available to your chart, ensuring that:

- **Consistency**: All charts use the same helper functions and naming conventions.
- **Maintainability**: Updates to shared templates propagate automatically to all dependent charts when dependencies are updated.
- **Standardization**: Organizational best practices are encoded in reusable templates, making it easier for team members to follow established patterns.
- **Efficiency**: Common operations and patterns are implemented once in the library and reused everywhere, reducing duplication and the potential for errors.

This approach allows your organization to maintain a consistent Helm chart structure and deployment patterns while still allowing individual charts the flexibility to customize as needed.
The exercises will demonstrate how to create and use library charts effectively.

Next Steps

We kept this section brief to focus on the practical use of templates.
Let us hop into the exercises where you will get hands-on experience creating templates, using functions, implementing flow control, and building reusable library charts.

Exercises

Exercise 1: Creating and Using Helper Functions

In this exercise, you will create a helper function (named template) and use it to generate a Kubernetes resource.
Helper functions allow you to encapsulate common patterns and reuse them throughout your chart.

Step 1: Create a New Chart

Create a new Helm chart for this exercise:

```
helm create helper-chart
```

Step 2: Create a Helper Function

Navigate to the templates directory and create a new file for your helper function:

```
cd helper-chart\templates
New-Item -Path "_helpers.tpl" -ItemType File
```

Open `_helpers.tpl` and add a helper function that generates standard labels.
Add the following content:

```
{{- define "helper-chart.labels" -}}
helm.sh/chart: {{ include "helper-chart.chart" . }}
{{ include "helper-chart.selectorLabels" . }}
{{- if .Chart.AppVersion }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
{{- end }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}
```

This helper function creates a set of standard labels that can be reused across multiple resources.

Step 3: Create a Resource Using the Helper

Create a new template file that uses your helper function to create a ConfigMap:

```
New-Item -Path "configmap.yaml" -ItemType File
```

Open `configmap.yaml` and add the following content:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "helper-chart.fullname" . }}-config
  labels:
{{ include "helper-chart.labels" . | indent 4 }}
data:
  app.properties: |
    environment={{ .Values.environment | default "development" }}
    logLevel={{ .Values.logLevel | default "info" }}
```

This ConfigMap uses the `helper-chart.labels` helper function to apply consistent labels.

Step 4: Test Your Template

Use `helm template` to render your chart and verify that the helper function works correctly:

```
cd ..\..
helm template test-release ./helper-chart
```

Review the output to confirm that:
* The ConfigMap is created with the correct name
* The labels from your helper function are properly applied
* The labels are correctly indented in the YAML output

Step 5: Verify the Helper Function Reusability

Create another resource (for example, a Secret) that also uses the same helper function:

```
cd helper-chart\templates
New-Item -Path "secret.yaml" -ItemType File
```

Add content to `secret.yaml`:

```
apiVersion: v1
kind: Secret
metadata:
  name: {{ include "helper-chart.fullname" . }}-secret
  labels:
{{ include "helper-chart.labels" . | indent 4 }}
type: Opaque
data:
  password: {{ .Values.password | b64enc }}
```

Render the chart again to see both resources using the same helper function:

```
cd ..\..
helm template test-release ./helper-chart
```

Notice how both the ConfigMap and Secret now have identical labels, demonstrating how helper functions enable consistency across resources while eliminating duplication.

Exercise 2: Using Helper Functions from Dependencies

In this exercise, you will create a new chart and add the helper chart from Exercise 1 as a dependency.
You will then use the helper function from the dependency in your new chart, demonstrating how library charts enable code reuse across multiple charts.

Step 1: Create a New Chart

Create a new Helm chart that will use the helper chart as a dependency:

```
cd ..\..
helm create app-chart
```

Step 2: Package the Helper Chart

First, package the helper chart so it can be used as a dependency.
If you are using a local file path, you can reference it directly, or package it for distribution:

```
helm package ./helper-chart
```

This creates a `helper-chart-0.1.0.tgz` file (assuming version 0.1.0 in Chart.yaml).

Step 3: Add the Helper Chart as a Dependency

Edit the `Chart.yaml` file in your `app-chart` directory to add the helper chart as a dependency:

```
cd app-chart
notepad Chart.yaml
```

Add a `dependencies` section to your `Chart.yaml`:

```
dependencies:
  - name: helper-chart
    version: "0.1.0"
    repository: "file://../helper-chart"
```

If you packaged the chart, you can also reference it from a local directory or OCI registry.

Step 4: Update Dependencies

Download the dependency into your chart:

```
helm dependency update
```

This downloads the helper chart into the `charts/` directory and makes its template helpers available to your chart.

Step 5: Use the Helper Function from the Dependency

Create a new resource in your `app-chart` that uses the helper function from the dependency:

```
cd templates
New-Item -Path "deployment.yaml" -ItemType File
```

Add content to `deployment.yaml` that uses the helper function from the dependency:

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: {{ include "app-chart.fullname" . }}
  labels:
{{ include "helper-chart.labels" . | indent 4 }}
spec:
  replicas: {{ .Values.replicaCount | default 1 }}
  selector:
    matchLabels:
      app.kubernetes.io/name: {{ include "app-chart.name" . }}
      app.kubernetes.io/instance: {{ .Release.Name }}
  template:
    metadata:
      labels:
{{ include "helper-chart.labels" . | indent 8 }}
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default
.Chart.AppVersion }}"
        imagePullPolicy: {{ .Values.image.pullPolicy }}
```

Notice that this Deployment uses `helper-chart.labels` from the dependency, not from the current chart.
The helper function is accessible because it is defined in a dependency.

Step 6: Test the Chart with Dependency

Render your chart to verify that the helper function from the dependency works correctly:

```
cd ..\..
helm template test-app ./app-chart
```

Review the output to confirm that:
* The Deployment is created with the correct name
* The labels from the helper-chart dependency are properly applied
* The helper function from the dependency is accessible and working

This demonstrates how library charts enable you to share helper functions across multiple charts, creating consistency and eliminating duplication at an organizational level.

Exercise 3: Type Checking and Validation

In this exercise, you will implement type checking to ensure that required values are defined and are of the correct type.
You will use Helm template functions to validate that a value is defined as a string before using it in your templates.

Step 1: Create a New Chart

Create a new Helm chart for this exercise:

```
cd ..\..
helm create type-check-chart
```

Step 2: Add Type Checking to a Template

Navigate to the templates directory and edit the `deployment.yaml` file:

```
cd type-check-chart\templates
notepad deployment.yaml
```

Modify the deployment to include type checking for a required string value.
Add a section that requires an `environment` value to be defined as a string:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "type-check-chart.fullname" . }}
  labels:
    {{- include "type-check-chart.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount | default 1 }}
  selector:
    matchLabels:
      app.kubernetes.io/name: {{ include "type-check-chart.name" . }}
      app.kubernetes.io/instance: {{ .Release.Name }}
  template:
    metadata:
      labels:
        {{- include "type-check-chart.labels" . | nindent 8 }}
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default
.Chart.AppVersion }}"
        imagePullPolicy: {{ .Values.image.pullPolicy }}
        env:
        - name: ENVIRONMENT
          value: {{ required "environment must be defined as a string"
.Values.environment | typeOf | eq "string" | ternary .Values.environment (fail
"environment must be a string, not " (typeOf .Values.environment)) }}
```

This example uses `required` to ensure the value exists, but we need a better approach for type checking.

Step 3: Create a Helper Function for String Type Checking

Create a helper function that validates a value is a string.
Edit or create `_helpers.tpl`:

```
notepad _helpers.tpl
```

Add a helper function that checks if a value is a string:

```
{{- define "type-check-chart.requireString" -}}
{{- if . -}}
  {{- if eq (typeOf .) "string" -}}
    {{- . -}}
  {{- else -}}
    {{- fail (printf "Value must be a string, got %s" (typeOf .)) -}}
  {{- end -}}
{{- else -}}
  {{- fail "Value is required and must be a string" -}}
{{- end -}}
{{- end -}}
```

Step 4: Use the Type Checking Helper

Update your `deployment.yaml` to use the helper function:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "type-check-chart.fullname" . }}
  labels:
    {{- include "type-check-chart.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount | default 1 }}
  selector:
    matchLabels:
      app.kubernetes.io/name: {{ include "type-check-chart.name" . }}
      app.kubernetes.io/instance: {{ .Release.Name }}
  template:
    metadata:
      labels:
        {{- include "type-check-chart.labels" . | nindent 8 }}
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default
.Chart.AppVersion }}"
        imagePullPolicy: {{ .Values.image.pullPolicy }}
        env:
```

```
        - name: ENVIRONMENT
          value: {{ include "type-check-chart.requireString" .Values.environment |
quote }}
```

Step 5: Test with Valid String Value

First, test the chart with a valid string value.
Edit `values.yaml`:

```
cd ..
notepad values.yaml
```

Add an environment value:

```
environment: "production"
```

Render the chart to verify it works:

```
helm template test-release ./type-check-chart
```

The chart should render successfully with the environment value set to "production".

Step 6: Test Type Validation

Now test that the type checking works by providing an invalid type.
Modify `values.yaml` to use a non-string value:

```
environment: 123
```

Attempt to render the chart:

```
helm template test-release ./type-check-chart
```

The template should fail with an error message indicating that the value must be a string, demonstrating that your type checking is working correctly.

This exercise demonstrates how to implement type checking in Helm templates, ensuring that values meet your requirements before they are used in your Kubernetes resources.

Exercise 4: Creating Resources from Array Combinations

In this exercise, you will use nested loops to create a ConfigMap for each possible combination of two arrays.
This demonstrates how to use `range` to iterate over multiple arrays and generate resources

dynamically.

Step 1: Create a New Chart

Create a new Helm chart for this exercise:

```
cd ..\..
helm create combination-chart
```

Step 2: Define Arrays in Values

Edit the `values.yaml` file to define two arrays that will be combined:

```
cd combination-chart
notepad values.yaml
```

Add two arrays to your values file:

```
environments:
  - name: development
    region: us-east-1
  - name: staging
    region: us-west-2
  - name: production
    region: eu-central-1

configurations:
  - type: standard
    replicas: 2
  - type: high-availability
    replicas: 5
```

This defines two arrays: `environments` and `configurations`.
We will create a ConfigMap for each combination of environment and configuration.

Step 3: Create a Template with Nested Loops

Navigate to the templates directory and create a template that generates ConfigMaps for each combination:

```
cd templates
New-Item -Path "configmaps.yaml" -ItemType File
```

Add content that uses nested `range` loops to create a ConfigMap for each combination:

```
{{- range $env := .Values.environments }}
```

```
{{- range $config := .Values.configurations }}
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ $.Release.Name }}-{{ $env.name }}-{{ $config.type }}-config
  labels:
    environment: {{ $env.name }}
    region: {{ $env.region }}
    config-type: {{ $config.type }}
    app.kubernetes.io/name: {{ include "combination-chart.name" $ }}
    app.kubernetes.io/instance: {{ $.Release.Name }}
data:
  environment: {{ $env.name | quote }}
  region: {{ $env.region | quote }}
  config-type: {{ $config.type | quote }}
  replicas: {{ $config.replicas | quote }}
{{- end }}
{{- end }}
```

This template uses nested `range` loops:
* The outer loop iterates over `environments`
* The inner loop iterates over `configurations`
* For each combination, it creates a ConfigMap with a unique name and data

Note the use of `$` to access the root context when inside nested loops.

Step 4: Test the Template

Render the chart to see all the ConfigMaps that are created:

```
cd ..\..
helm template test-release ./combination-chart
```

Review the output to verify that:
* A ConfigMap is created for each combination of environment and configuration
* Each ConfigMap has a unique name combining the environment name and configuration type
* The data in each ConfigMap reflects the values from both arrays
* You should see 6 ConfigMaps total (3 environments × 2 configurations)

Step 5: Verify the Combinations

Count the ConfigMaps in the output to confirm all combinations are created:

```
helm template test-release ./combination-chart | Select-String -Pattern "kind:
ConfigMap" | Measure-Object
```

You should see 6 ConfigMaps, one for each combination:

* development-standard
* development-high-availability
* staging-standard
* staging-high-availability
* production-standard
* production-high-availability

This exercise demonstrates how to use nested loops in Helm templates to generate multiple resources from array combinations, enabling you to create complex deployment scenarios efficiently.

Module 4: Storage and Distribution of Charts

Content

Once you have created and tested your Helm charts, you need to store and distribute them so they can be used across your organization or shared with others.
Helm supports two primary methods for storing and distributing charts: traditional chart repositories and OCI (Open Container Initiative) registries.

Chart Repositories

Traditional Helm chart repositories are HTTP servers that host chart packages and an `index.yaml` file containing metadata about available charts.
Repositories provide a centralized location for storing and discovering charts.

Repository Structure

A chart repository typically contains:
* Chart package files (`.tgz` files)
* An `index.yaml` file that lists all charts, their versions, and metadata
* Optional chart documentation and README files

Working with Repositories

You can add repositories, search for charts, and install from repositories using standard Helm commands.
Repositories are well-suited for organizations that want to maintain their own chart hosting infrastructure.

OCI Registries

Helm 3.8+ supports storing charts as OCI artifacts in container registries.
This allows you to use the same registries you use for container images to store Helm charts, simplifying your infrastructure.

OCI Artifacts

When you push a chart to an OCI registry, Helm packages it as an OCI artifact.
OCI registries provide:
* Authentication and authorization using existing registry credentials
* Versioning and tagging similar to container images
* Integration with existing container registry infrastructure
* Support for both public and private registries

Advantages of OCI Registries

Using OCI registries for Helm charts offers several advantages:
* **Unified Infrastructure**: Use the same registry for both container images and Helm charts

* **Simplified Authentication**: Leverage existing registry authentication mechanisms
* **Better Integration**: Works seamlessly with GitOps tools and CI/CD pipelines
* **Modern Standard**: OCI is the modern standard for artifact distribution

Packaging Charts

Before distributing a chart, you package it into a compressed archive:

```
helm package ./my-chart
```

This creates a `.tgz` file containing all chart files.
The package can then be uploaded to a repository or pushed to an OCI registry.

Signing and Verifying Charts

Chart signing provides cryptographic verification that a chart package has not been tampered with and comes from a trusted source.
This is essential for maintaining security and integrity in your chart distribution pipeline.

Why Sign Charts?

Signing charts ensures:
* **Integrity**: Verifies that the chart has not been modified since it was signed
* **Authenticity**: Confirms that the chart comes from a trusted publisher
* **Non-repudiation**: Provides proof of who created and signed the chart
* **Security**: Protects against supply chain attacks and unauthorized modifications

Signing a Chart

To sign a chart, you need a PGP key pair.
Once you have a key, sign a packaged chart:

```
helm package ./my-chart
helm sign my-chart-0.1.0.tgz --key <key-name>
```

This creates a `.tgz.prov` (provenance) file containing the signature and metadata about the chart.

Verifying a Signed Chart

Before installing a chart, verify its signature to ensure it has not been tampered with:

```
helm verify my-chart-0.1.0.tgz
```

This command checks that:
* The signature file exists
* The signature is valid

* The chart matches the signed version
* The signing key is trusted

If verification fails, Helm will not install the chart, protecting you from potentially compromised packages.

Key Management

For production use, establish a key management strategy:
* Store private keys securely (key vaults, hardware security modules)
* Distribute public keys to chart consumers
* Rotate keys periodically
* Maintain a keyring of trusted public keys

Chart signing is a critical security practice, especially when distributing charts outside your organization or when charts are used in production environments.

Distribution Methods

Choose your distribution method based on your organization's needs:

- **Chart Repositories**: Best for organizations that want dedicated chart hosting with full control over the repository structure and metadata.
- **OCI Registries**: Best for organizations already using container registries and wanting to consolidate artifact storage, or for teams adopting modern cloud-native practices.

Both methods are valid and supported by Helm.
The choice depends on your infrastructure, security requirements, and organizational preferences.

Next Steps

The exercises will guide you through packaging charts, pushing them to OCI registries, and working with both distribution methods.
You will gain hands-on experience with the commands and workflows needed to distribute your charts effectively.

Exercises

Exercise 1: Packaging a Chart as an OCI Image and Examining the Artifact

In this exercise, you will package a Helm chart and push it to an OCI registry, then use Docker to examine the resulting OCI artifact.
This demonstrates how Helm charts are stored as OCI artifacts and how you can inspect them using standard container tools.

Step 1: Create and Package a Chart

Create a new Helm chart for this exercise:

```
helm create oci-chart
```

Package the chart:

```
helm package ./oci-chart
```

This creates a `oci-chart-0.1.0.tgz` file (assuming version 0.1.0 in Chart.yaml).

Step 2: Push the Chart to an OCI Registry

Before pushing, ensure you are logged into your OCI registry.
If you are using a local registry or need to log in:

```
helm registry login <registry-url>
```

Push the packaged chart to an OCI registry:

```
helm push oci-chart-0.1.0.tgz oci://<registry-url>/charts
```

For example, if using a local registry:

```
helm push oci-chart-0.1.0.tgz oci://localhost:5000/charts
```

This uploads your chart as an OCI artifact to the registry.

Step 3: Examine the OCI Artifact with Docker

Use Docker to pull and examine the OCI artifact.
First, pull the artifact:

```
docker pull <registry-url>/charts/oci-chart:0.1.0
```

For a local registry example:

```
docker pull localhost:5000/charts/oci-chart:0.1.0
```

Step 4: Inspect the OCI Artifact

Examine the manifest of the OCI artifact:

```
docker manifest inspect <registry-url>/charts/oci-chart:0.1.0
```

This shows the OCI manifest structure, including:
* Media types
* Layers
* Annotations
* Configuration

Step 5: Inspect the Image Details

Get detailed information about the OCI artifact:

```
docker image inspect <registry-url>/charts/oci-chart:0.1.0
```

This provides comprehensive information about the artifact, including:
* Architecture
* Created timestamp
* Environment variables
* Labels and annotations
* Layer information

Step 6: View the Artifact Layers

Examine the layers that make up the OCI artifact:

```
docker history <registry-url>/charts/oci-chart:0.1.0
```

This shows the layer history, helping you understand how the chart is structured as an OCI artifact.

This exercise demonstrates that Helm charts stored as OCI artifacts can be examined using standard Docker commands, showing how Helm integrates with the OCI ecosystem.